

Aim:**Understanding conversion of infix to postfix expression**

Write the code in the functions `convertInfixToPostfix (self,expr)`, `push(self, value)`, `pop(self)`, `peek(self)`, `isOperand(self, i)` and `isEmpty(self)` in the below code according to the hints given as comment lines.

Sample Output -1

Enter a valid infix expression where operator is placed between the operands: A+B*(C+D)

The postfix expression of : A+B*(C+D) is ABCD+*+

Sample Output -2

Enter a valid infix expression where operator is placed between the operands: A+D*C+E-F&D

Incorrect infix expr

Sample Output -3

Enter a valid infix expression where operator is placed between the operands: A+B*C+(D*E

Incorrect infix expr

Source Code:**Infix2Postfix.py**

```
'''
Class infixToPostfix defined with constructor, push, pop, isEmpty, seek
isOperand and convertInfixToPostfix functionalities.
'''
class infixToPostfix:
    precedence={'^':5,'*':4,'/':4,'+':3,'-':3,'(':2,')':1}
    def __init__(self):
        self.stack = []
        self.size = -1

    def push(self, value):
        self.stack.append(value)

    def pop(self):
        if len(self.stack) == 0:
            print("Stack is empty.")
        else:
            return self.stack.pop()

    def isEmpty(self):
        if len(self.stack) == 0:
            return True
        else:
            return False
```

```

def peek(self):
    if len(self.stack) != 0:
        return self.stack[len(self.stack)-1]
    else:
        print("Stack is empty.")

def isOperand(self, i):
    valid_operands =
["a","b","c","d","e","f","g","h","i","j","l","m","n","o","p","q","r","s","t","u","v",
"w","x","y","z","A","B","C","D","E","F","G","H","I","J","K","L","M","N","O","P","Q","
R","S","T","U","V","W","X","Y","Z"]
    if i in valid_operands:
        return True
    else:
        return False

def convertInfixToPostfix (self,expr):
    postfix_expr = ""
    valid_symbols = ["+", "-", "*", "/", "^", "(", ")"]

    if (expr.count('(')) != (expr.count(''))):
        return False

    for i in set(expr)-set(valid_symbols):
        if self.isOperand(i):
            pass
        else:
            return False

    for i in expr:
        if self.isOperand(i):
            postfix_expr += str(i)
        elif i == "(":
            self.push(i)
        elif i == ")":
            while self.stack and self.peek() != "(":
                postfix_expr += self.pop()
            self.stack.pop()
        else:
            while self.stack and self.precedence[self.peek()] >= self.precedence[i]:
                postfix_expr += self.pop()
            self.push(i)

    while self.stack:
        postfix_expr += self.pop()

    return postfix_expr

# Driver code to test all functionalities of class infixToPostfix
ip=infixToPostfix()
expr=input('Enter a valid infix expression where operator is placed between the opera
nds: ')
result=ip.convertInfixToPostfix(expr)
if (result!=False):
    print("The postfix expression of :",expr,"is",result)

```

```
else:  
    print("Incorrect infix expr")
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Enter a valid infix expression where operator is placed between the operands: A+B*(C+D)
The postfix expression of : A+B*(C+D) is ABCD+*+

Test Case - 2
User Output
Enter a valid infix expression where operator is placed between the operands: A+D*C+E-F&D
Incorrect infix expr

Test Case - 3
User Output
Enter a valid infix expression where operator is placed between the operands: A+B*C+(D*E
Incorrect infix expr

Test Case - 4
User Output
Enter a valid infix expression where operator is placed between the operands: (A+B
Incorrect infix expr

Test Case - 5
User Output
Enter a valid infix expression where operator is placed between the operands: A@B\$C
Incorrect infix expr